

Modelling humanities data with TEI-XML

SCHOLARLY EDITING AND MANUSCRIPT CATALOGUING IN THE DIGITAL AGE

Dr Katarzyna Anna Kapitan
10 December 2025

Recap: ODD

```
<TEI xml:lang="en">  
  <teiHeader> Metadata </teiHeader>  
  <text>  
    <body>  
      <p> Description of the schema </p>  
      <schemaSpec ident="test" targetLang="en"> [...]  
        <moduleRef key="module_name"/>  
        <elementSpec ident="element_name" mode="mode_of_change">  
          [...]  
        </elementSpec></schemaSpec></body></text></TEI>
```

Recap ODD: Week9, Exercise 3

- ▶ Create an ODD file from **minimal_encoding.xml** by using the **oddbysample.xsl**.
 - ▶ Tutorial: Burnard_2013_How_to_Make_an_ODD_Automagically.pdf
- ▶ Generate **RELAX NG** schema and **XHTML** guidelines from your ODD.
- ▶ Associate the RNG schema with your XML file (**minimal_encoding.xml**),
 - ▶ Does it validate correctly?
 - ▶ What if you tag “Paris” with **placeName**?
- ▶ Edit the ODD file so that it additionally allows the **placeName** element in the main body of the text.
- ▶ Document this change by providing a brief **description** in **<desc>** and one usage **example** in **<exemplum>** **<egXML>** (See ODD from Exercise 1 for inspiration).

TEI	teiHeader	fileDesc	titleStmt	title
1	<?xml version="1.0" encoding="UTF-8"?>			
2	<?xml-model href="out/myGenerated.rng" type="application/xml" schematypens="http://relaxng.org/ns/structure/1.0"?>			
3	<TEI xmlns="http://www.tei-c.org/ns/1.0">			
4	<teiHeader>			
5	<fileDesc><titleStmt><title><!-- My title --></title></titleStmt>			
6	<publicationStmt>			
7	<authority>Katarzyna Anna Kapitan</authority>			
8	<availability>			
9	<licence>CC-BY 4.0</licence>			
10	</availability>			
11	</publicationStmt>			
12	<sourceDesc>			
13	<p>Born digital, created on <date when="2024-11-28">28/11/2024</date> to test the			
14	oddbyeexample transformation</p>			
15	</sourceDesc>			
16	</fileDesc>			
17	<revisionDesc><listChange><change><!-- Info about my revision history --></change></listChange></revisionDesc>			
18	</teiHeader>			
19	<text><front> <!-- Some text --> </front>			
20	<body>			
21	<div>			
22	<p>Here comes my text, it's about the TEI XML course taught by <persName>Katarzyna			
23	Kapitan</persName> in <placeName>Paris</placeName>.</p>			
24	</div>			
25	</body>			
26	</back>			

Custom schema
(generated with xsl
oddbyeexample)
associated with the file.

❗ element "placeName" not allowed anywhere; expected the element end-tag, text or element "date", "persName" or "title"

Validation error when
placeName element
added

TEI: Guidelines for Electronic Text Encoding and Interchange

P5 Version 4.10.2. Last updated on 4th September 2025, revision bcfa98f42

<placeName>

Home
C Elements

<placeName> (place name) contains an absolute or relative place name. [14.2.3 Place Names]	
Module	namesdates — Names, Dates, People, and Places
Attributes	- att.global: @xml:id, @n, @xml:lang, @xml:base, @xml:space - att.global.analytic: @ana - att.global.change: @change - att.global.facs: @facs - att.global.linking: @corresp, @synch, @sameAs, @copyOf, @next, @prev, @exclude, @select - att.global.rendition: @rend, @style, @rendition - att.global.responsibility: @cert, @resp - att.global.source: @source - att.cmc: @generatedBy - att.dateable: @period - att.dateable.custom: @when-c @datingMethod

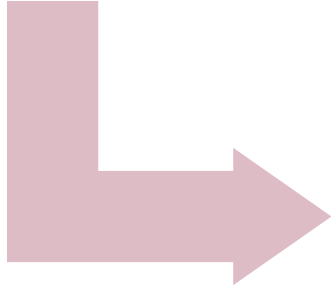
<desc> Your project-specific description.

<egXML> An encoding example from our project.

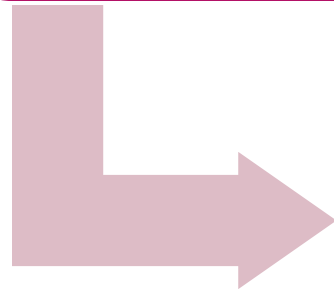
```
<!-- Adding placeName with a project-specific description and an example -->
<moduleRef key="namesdates" include="persName placeName"/>
<elementSpec ident="persName" mode="change">
  <attList>
    <attDef ident="when" mode="delete"/>
  </attList>
</elementSpec>
<elementSpec ident="placeName" mode="change">
  <desc> [Note Change!] contains an absolute or relative place name. In our
    project we use elements such as <gi>placeName</gi> and <gi>persName</gi> within
    the body of text (rather than <gi>name</gi> with type). </desc>
  <exemplum>
    <egXML xmlns="http://www.tei-c.org/ns/Examples">
      <placeName>Paris</placeName>
    </egXML>
  </exemplum>
</elementSpec>
```



- Customisation & Documentation
- Guidelines & Framework for your encoding project



- RELAX NG Schema
- Set of formal rules to control your encoding
- Generated from ODD



- Your valid TEI-XML document with your consistently encoded data

Schematron

Schematron

- ▶ Is a formal schema language to express rules for XML documents.
- ▶ Is a relatively simple but powerful validation language
- ▶ Is expressed in XML
- ▶ Relies heavily on **XPath**, which means that if you want to write **Schematron** schemas, you have to understand **XPath** (Week 8)
- ▶ Source: Erik Siegel, *Schematron: A language for validating XML* (2022)



Join this Wooclap event



1

Go to wooclap.com

2

Enter the event code in the
top banner

Event code

HTZMFP



Enable answers by SMS

XPath Quiz

Schematron

- ▶ In Schematron you define all the **error** and **report** messages in your own words!
- ▶ There are two types of Schematron rules:
 - ▶ **Assertions:** when the condition for an assertion fails, an error message is issued.
 - ▶ **Reports:** when the condition for a report hold, a report message is issued.

Testing for presences/absence of attributes: Assertion

```
<sch:assert test="@type">
```

The type attribute is required.

```
</sch:assert>
```

- ▶ Make sure that the type attribute (@type) is present
- ▶ When there is **NO** @type, notify me!

Testing for presences/absence of attributes: Report

```
<sch:report test="not(@type)" role="error">
```

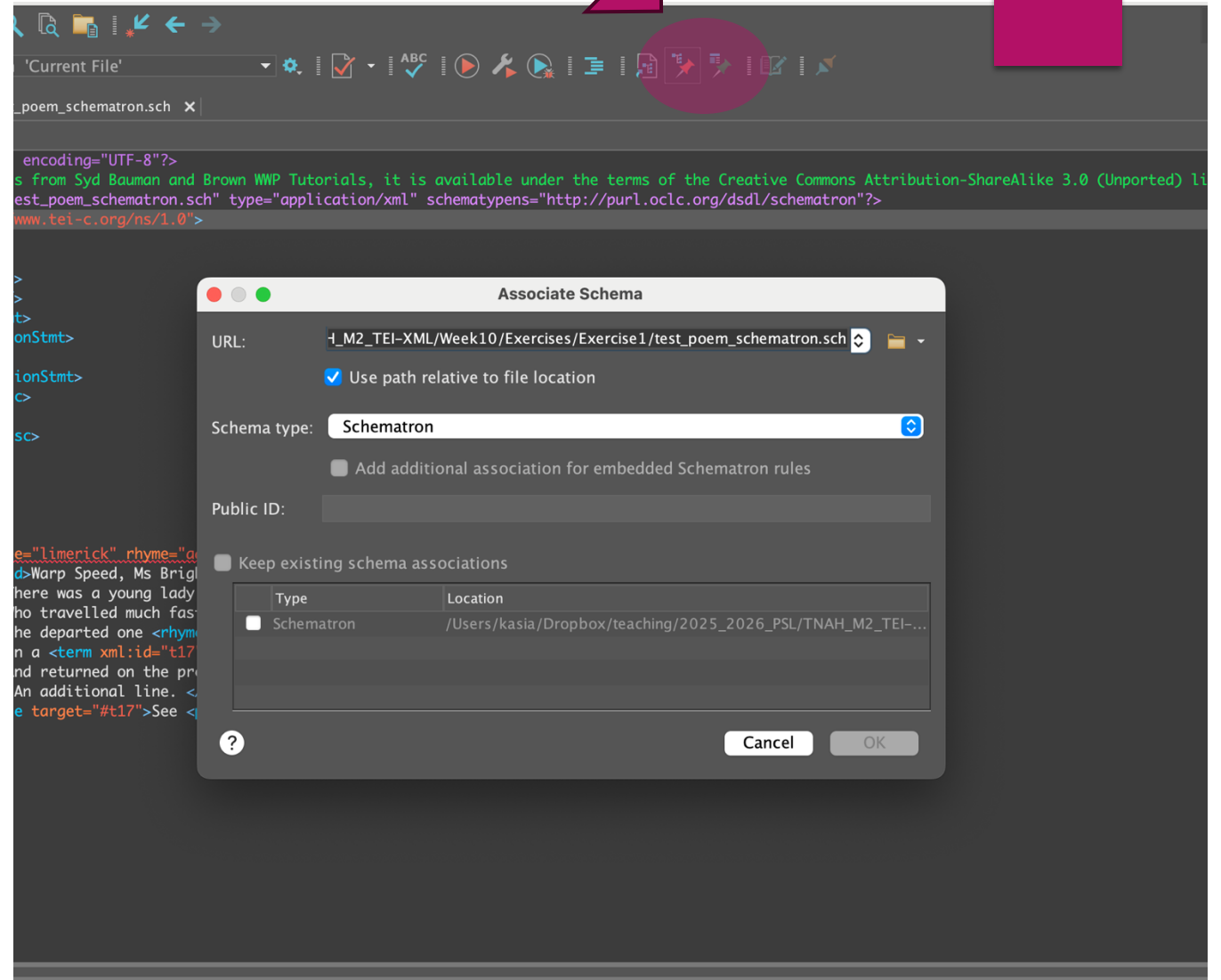
The type attribute is required.

```
</sch:report >
```

- ▶ Check whether the type attribute (@type) is present
- ▶ When there is **NO** @type, give me an error!

Validating XML with stand-alone schematron

- ▶ You can validate your XML files with stand-alone schematron files (.sch)
- ▶ You associate the schema in the same way as RelaxNG schema.



Stand-alone Schematron & name spaces

```
▶ <?xml version="1.0" encoding="UTF-8"?>
<sch:schema xmlns:sch=http://purl.oclc.org/dsdl/schematron
queryBinding="xslt2" xmlns:sqf="http://www.schematron-
quickfix.com/validator/process">
  <sch:ns uri="http://www.tei-c.org/ns/1.0" prefix="tei"/>
  <sch:pattern>
    <sch:rule>
      Here goes your rule which uses TEI elements
    </sch:rule>
  </sch:pattern></sch:schema>
```

Schematron Rules & XPath

In **Schematron**, the context item is set using **@context** attribute of the **rule** element.

```
<sch:rule context="tei:author" >  
  <sch:assert test="child::tei:surname" role="error">
```

Each author entry needs a surname

```
</sch:assert>  
</sch:rule>
```

Testing for presences/absence of elements as children

```
<sch:report test="not(child::tei:surname)" role="error">
```

The surname element is required.

```
</sch:report>
```

- Check whether there is **no surname** element as a child of my context node and give me an error message when this is the case.

Exercise 1: Stand-alone Schematron

- ▶ Open **test_poem_tei.xml** and **test_poem_schematron.sch**
- ▶ Associate the SCH schema with your XML, validate it.
- ▶ Fix the errors in XML, so it validates correctly.
- ▶ In SCH file make the **rhyme** attribute required for lg
- ▶ In XML delete the rhyme attribute from lg, check whether you get an error.

Schematron rules, Xpath, and error messages

```
<sch:rule context="tei:lg[@type='limerick']">  
  <sch:report test="count( child::tei:l ) != 5" role="error">
```

A limerick should not have

```
<sch:value-of select="count( child::tei:l )"/>
```

lines.

```
</sch:report>
```

```
</sch:rule>
```

Assertion or Report – Your Choice!

Assertion:

```
<sch:assert test="count( child::tei:l ) = 5" role="error">
```

A limerick should have 5 lines

```
</sch:assert>
```

Report:

```
<sch:report test="count( child::tei:l ) != 5" role="error">
```

A limerick shouldn't have <sch:value-of select="count(child::tei:l)"/> lines.

```
</sch:report>
```

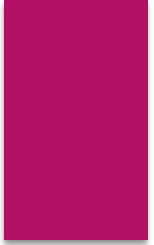
Schematron in ODD

Schematron in ODD

- ▶ You can use schematron with your ODD you need to define its namespace.
- ▶ You can do it on the TEI element or schemaSpec
- ▶ `<TEI xmlns="http://www.tei-c.org/ns/1.0" xml:lang="en" xmlns:sch="http://purl.oclc.org/dsdl/schematron">`

Schematron in ODD

- ▶ You can use schematron rules within **elementSpec**
- ▶ To do it you need two new elements **constraintSpec** & **constraint**:
- ▶ `<constraintSpec ident="identifier" mode="mode_of_change" scheme="isoschematron">`
 `<constraint>` Here goes your Schematron rule `</constraint>`
`</constraintSpec>`



```
<elementSpec ident="lg" mode="change">
```

```
  <constraintSpec ident="lg_limerick" mode="change"  
    scheme="isoschematron">
```

```
    <constraint>
```

```
      <sch:rule context=".[@type='limerick']">
```

```
        <sch:report test="count( child::tei:l ) != 5" role="error">
```

A limerick should not have

<sch:value-of select="count(child::tei:l)"/> lines.

It should have 5 lines. Fix your encoding.

```
        </sch:report>
```

```
      </sch:rule>
```

```
    </constraint>
```

```
  </constraintSpec>
```

```
</elementSpec>
```

Exercise 2: Schematron in ODD

- ▶ Using **oddbysample.xsl** & **test_poem_tei.xml** create an ODD file.
- ▶ Edit the ODD file to allow for schematron rules (namespace).
- ▶ Incorporate a schematron rule for **line group** with **@type 'limerick'** that gives an error when there the number of lines isn't equal to 5.
`<sch:report test="count( child::tei:l ) != 5" role="error"> [...]`
- ▶ *Where should this rule appear?*
- ▶ Export your ODD into RELAX NG.
- ▶ Validate **test_poem_tei.xml** with your new RELAX NG.
- ▶ *Do you get an error? You should!*


```

<elementSpec ident="lg" mode="change">
  <constraintSpec ident="lg_limerick" mode="change" scheme="isoschematron" >
    <desc>[Note Change!] We want to make sure that limericks always have only 5 lines.</desc>
    <constraint>
      <sch:rule context=".[@type='limerick']">
        <sch:report test="count(child::tei:l) !=5" role="error">
          A limerick should have 5 lines. Fix your encoding!
        </sch:report>
      </sch:rule>
    </constraint>
  </constraintSpec>

```

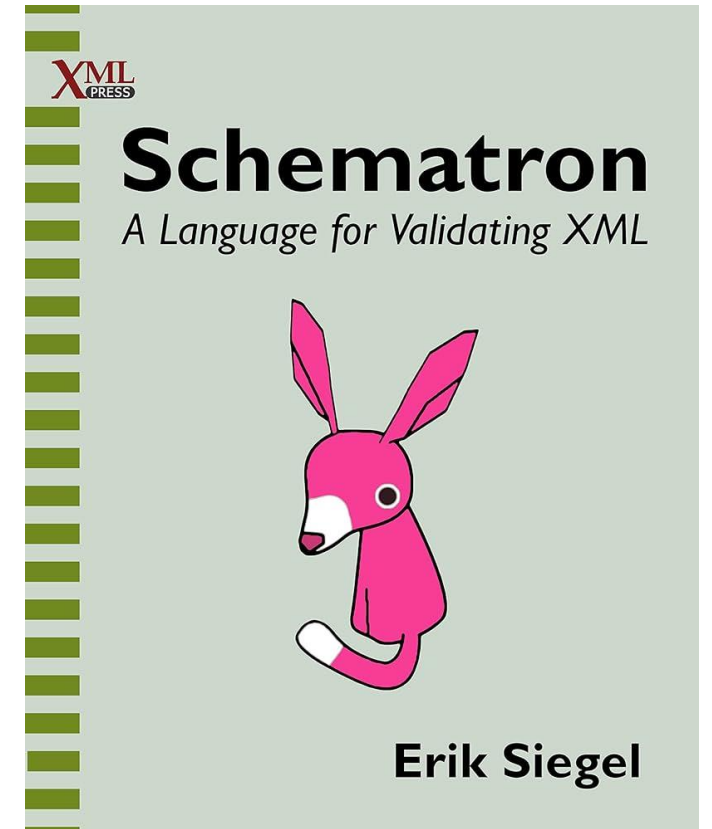
In your ODD

Schematron	[Note Change!] We want to make sure that limericks always have only 5 lines. <pre> <sch:rule context=".[@type='limerick']"> <sch:report test="count(child::tei:l) !=5" role="error"> A limerick should have 5 lines. Fix your encoding! </sch:report> </sch:rule> </pre>
-------------------	---

In your project guidelines (HTML)

More about Schematron

- ▶ Erik Siegel, *Schematron: A language for validating XML* (2022)



Assignment 3 (40%)

Submission Deadline: 9 January 2026 @ 23:59 (Paris time zone)

Submission Form: Folder on GitHub consisting of:

1. **ReadMe file** (100–150 words) describing your repository.
2. **ODD customisation file.**
3. **RNG and HTML files** generated from your ODD.
4. **TEI XML files:** your samples encoded using your customised schema, validated against your RNG.
5. **Reflexive note** (250–300 words) reflecting on your learning process throughout the project.

2. Customise your schema.

- Define and document at least three custom rules specific for your project:

Week 9

Weeks 8 & 10

- A rule constraining the use of an attribute and its value(s) (<attDef>).
- A rule constraining the sequence of certain elements (<content>).
- A rule constraining the value of an attribute or the use of an element/attribute depending on its environment (<constraint> + <sch:rule>).
- **Note:** The rules cannot be the same ones as those presented in the class.

• Provide descriptions of your rules in <desc> within your ODD. Use ‘ENote Change’ at

Encoding Portfolio: Assignment 3

NEW DEADLINE: 9 JANUARY!

Thank you for this term!!!

GOOD LUCK WITH THE FINAL SUBMISSION!!!